Санкт-Петербургский национальный исследовательский университет
информационных технологий, механики и оптики
Дисциплина: Фронт-энд разработка

ОТЧЁТ
по лабораторной работе № 3
Разработка приложения на Vue.js

Выполнил: Цатинян Артём
Группа: K3439

Санкт-Петербург
2026 г.

Цель работы

Перенести приложение tablely для поиска ресторанов и бронирования столиков на Vue.js, сохранив его внешний вид и основные пользовательские сценарии. Освоить компонентный подход, маршрутизацию и получение данных через HTTP API.

Задание

На основе приложения предыдущих лабораторных работ создать интерфейс на Vue.js, организовать переходы с помощью Vue Router, выполнять запросы к API и вынести повторно используемую логику в composable-функции.

Ход работы

1. Подготовка и структура проекта

Для третьей работы создан отдельный проект в каталоге lab3. Для разработки и сборки используется Vite; Vue 3 отвечает за интерфейс, Vue Router — за переходы, Pinia — за состояние авторизации, Axios — за HTTP-запросы. Визуальные стили и локальный сервер приложения перенесены из предыдущей работы.

Код разделён по назначению: views содержат страницы, components — переиспользуемые элементы, router — описание маршрутов, stores — состояние пользователя, api — HTTP-клиент, composables — общую реактивную логику.

2. Компоненты и реактивное состояние

Общие элементы вынесены в компоненты: AppHeader отвечает за навигацию и мобильное меню, RestaurantCard отображает карточку ресторана, SvgIcon выводит значки, ThemeToggle переключает тему. Страницы каталога, ресторана, авторизации и личного кабинета размещены отдельно.

На главной странице реактивные значения хранят выбранные кухню, район, дату и число гостей. Список ресторанов обновляется при изменении фильтров. Дата и количество гостей передаются на страницу выбранного ресторана через параметры адреса.

3. Маршрутизация

Vue Router связывает адреса /, /login, /register, /restaurants/:id и /profile с соответствующими представлениями. Для неизвестных адресов предусмотрена отдельная страница. Перед открытием личного кабинета проверяется состояние авторизации: гость перенаправляется на страницу входа, после чего может вернуться к исходному адресу.

4. Запросы к API и авторизация

Axios-клиент использует базовый адрес /api; сервер Vite перенаправляет запросы локальному API. Данные ресторанов загружаются запросами GET, а параметры фильтра передаются в строке запроса. Скриншот панели Network показывает запросы с параметрами кухни и района и ответы в формате JSON.

Для регистрации и входа отправляются POST-запросы. Полученный токен хранится в sessionStorage и добавляется к последующим запросам в заголовке Authorization. Pinia хранит данные текущего пользователя и восстанавливает сессию; доступ к данным бронирования проверяется сервером.

5. Бронирование и общая логика

На странице ресторана пользователь выбирает дату, время и число гостей. Авторизованный пользователь создаёт бронь; в личном кабинете можно открыть форму изменения или отменить будущую бронь. Для этих операций используются соответственно POST, PATCH и DELETE запросы к API.

Composable useAsync управляет состояниями загрузки и ошибки при выполнении запросов. useTheme отвечает за переключение светлой и тёмной темы и сохранение выбора в localStorage. Общая функция getLocalDate формирует текущую локальную дату для полей выбора.

6. Проверка приложения

Проверены загрузка каталога, фильтрация, переход к ресторану, регистрация и вход, создание, изменение и отмена бронирования. Дополнительно проверены защищённый маршрут личного кабинета, мобильная навигация и сохранение выбранной темы. Сборка Vite и два модульных теста завершились успешно; браузерный сценарий основных действий также прошёл.

Результаты работы

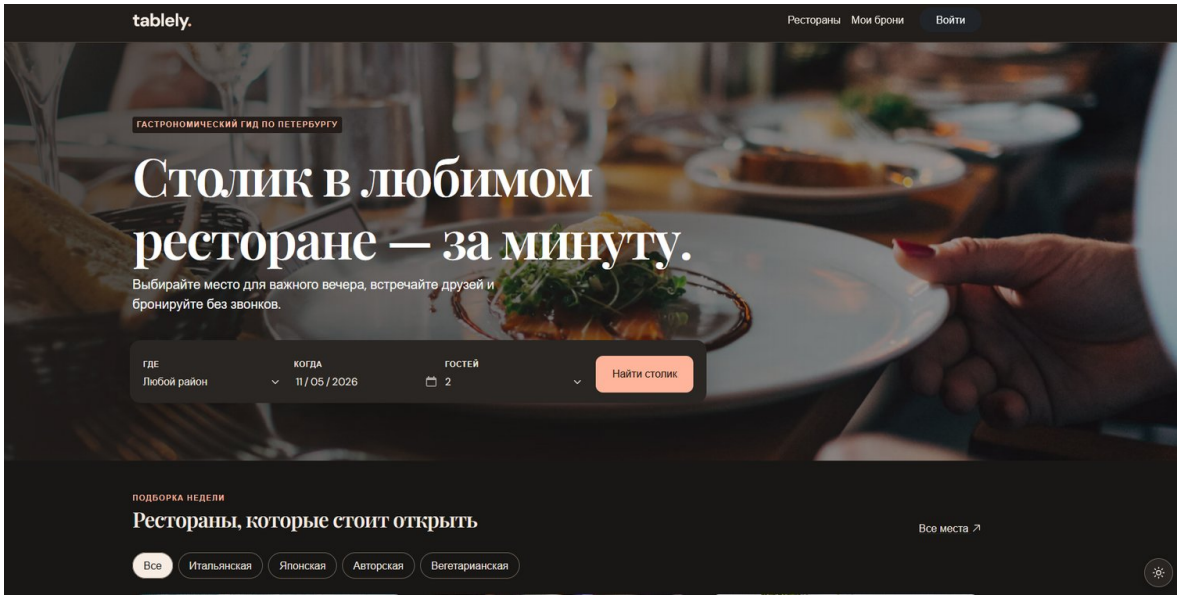


Рисунок 1 — Главная страница приложения и форма поиска

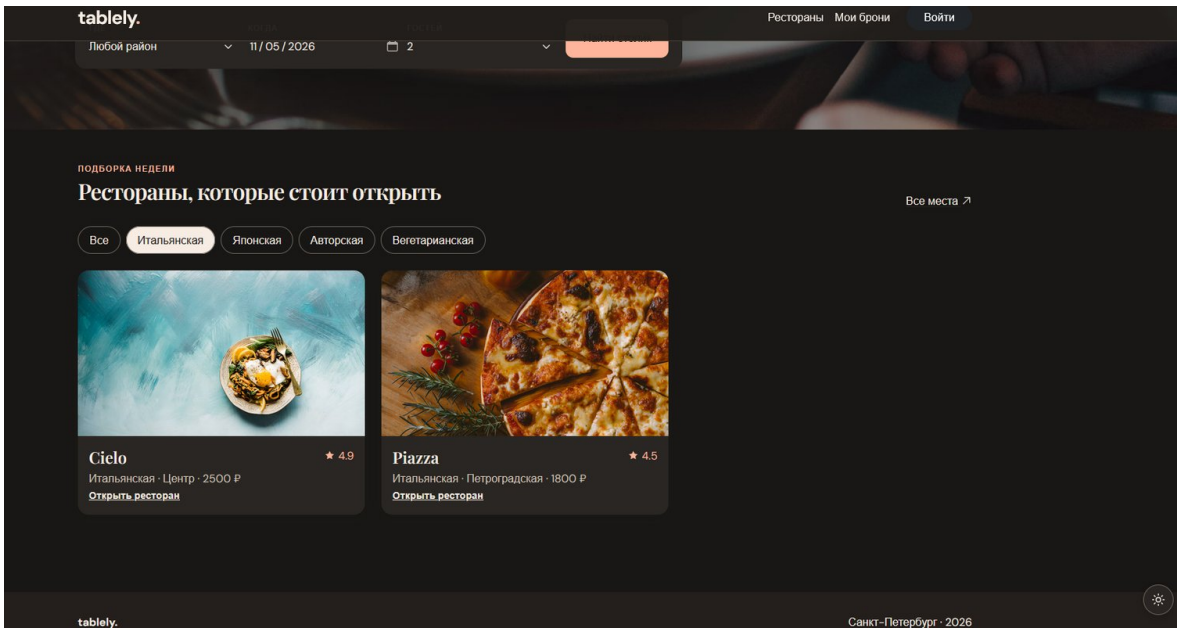


Рисунок 2 — Фильтрация ресторанов по итальянской кухне

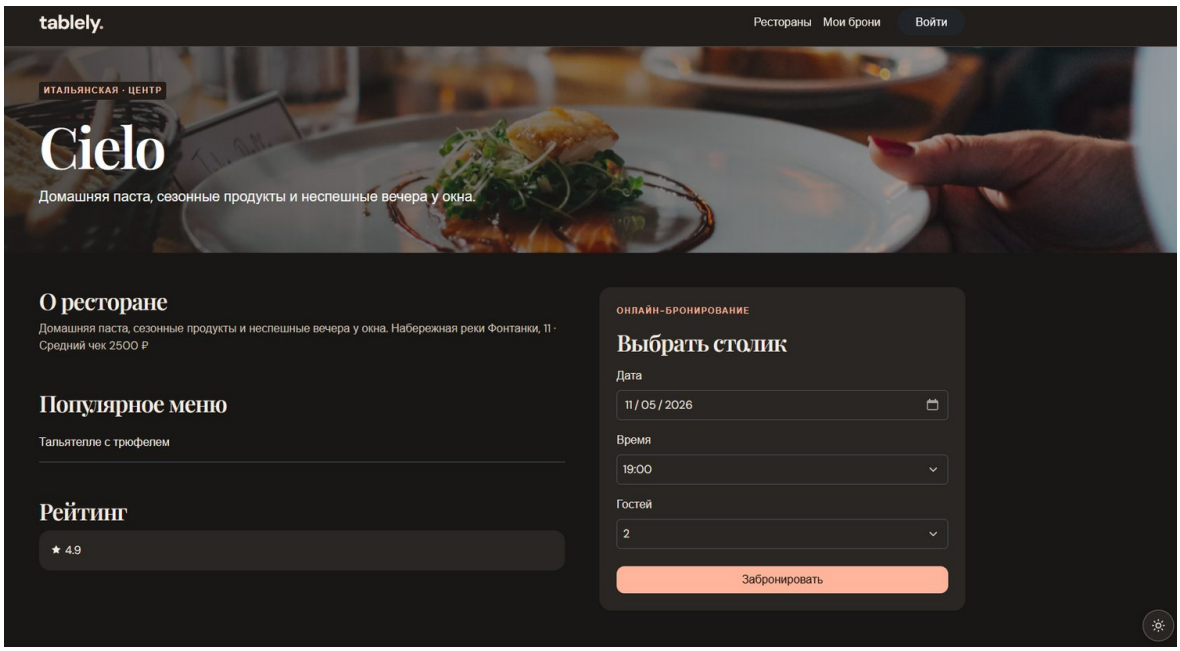


Рисунок 3 — Страница ресторана и форма бронирования

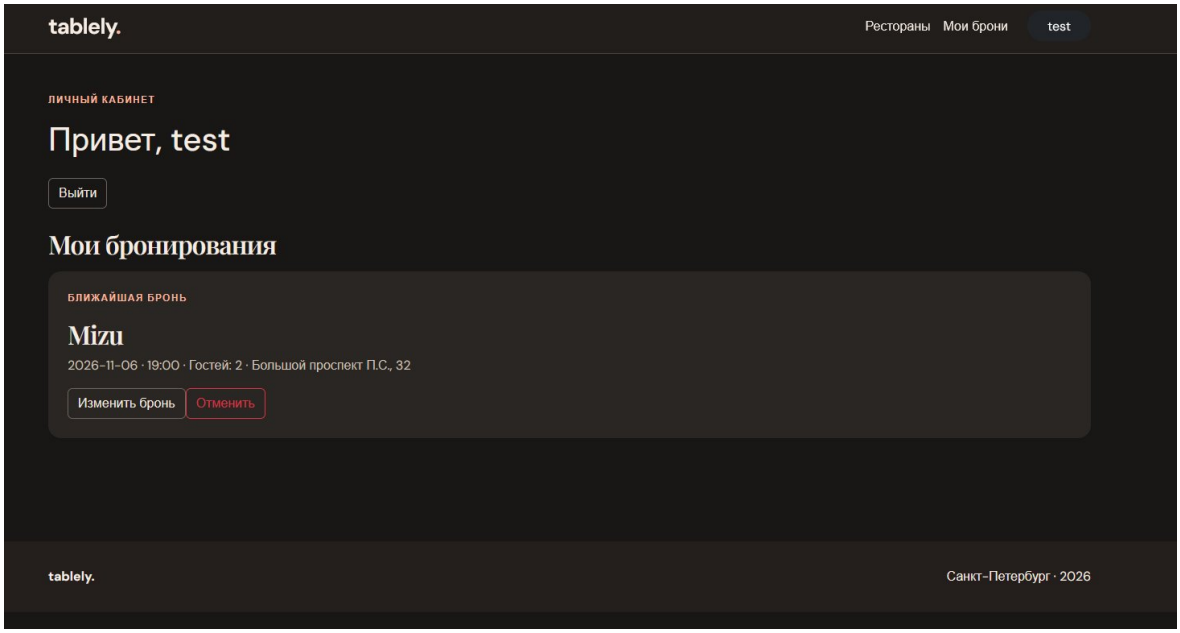


Рисунок 4 — Личный кабинет с созданной бронью

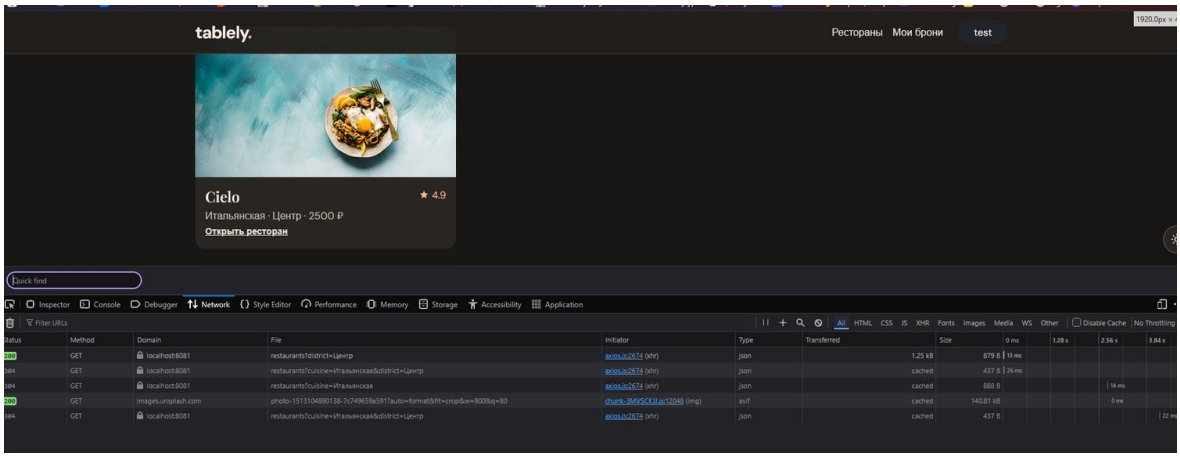


Рисунок 5 — Запросы Axios к API в панели Network

Вывод

В ходе работы приложение tablely перенесено на Vue.js. Реализованы компонентная структура, маршрутизация, общее состояние авторизации и взаимодействие с локальным API. Основные пользовательские сценарии проверены, результаты представлены на рисунках.